

6.2.1 Storing real numbers as floating point numbers



6.2.1 Storing reals as floats

fit width



- Only a finite number of (binary) digits can be used to store a real number in the memory of a computer. For so-called single-precision and double-precision floating point numbers, 32 bits and 64 bits are typically employed, respectively.
- Recall that any real number can be written as $\mu \times \beta^e$, where β is the base (an integer greater than one), $\mu \in (-1, 1)$ is the mantissa, and e is the exponent (an integer). For our discussion, we will define the set of floating point numbers, F , as the set of all numbers $\chi = \mu \times \beta^e$ such that
- $\beta = 2$,
 - $\mu = \pm .\delta_0\delta_1 \cdots \delta_{t-1}$ (μ has only t (binary) digits), where $\delta_j \in \{0, 1\}$,
 - $\delta_0 = 0$ iff $\mu = 0$ (the mantissa is normalized), and
 - $-L \leq e \leq U$.
- With this, the elements in F can be stored with a finite number of (binary) digits.

Example 6.2.1.1. Let $\beta = 2$, $t = 3$, $\mu = .101$, and $e = 1$. Then

$$\begin{aligned} \mu \times \beta^e &= \\ .101 \times 2^1 &= \\ (1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}) \times 2^1 &= \\ \left(\frac{1}{2} + \frac{0}{4} + \frac{1}{8}\right) \times 2 &= \\ 1.25 \end{aligned}$$

- Observe that
- There is a largest number (in absolute value) that can be stored. Any number with larger magnitude "overflows". Typically, this causes a value that denotes a NaN (Not-a-Number) to be stored.
 - There is a smallest number (in absolute value) that can be stored. Any number that is smaller in magnitude "underflows". Typically, this causes a zero to be stored.

In practice, one needs to be careful to consider overflow and underflow. The following example illustrates the importance of paying attention to this.

Example 6.2.1.2. Computing the (Euclidean) length of a vector is an operation we will frequently employ. Careful attention must be paid to overflow and underflow when computing it.

Given $x \in \mathbb{R}^n$, consider computing

$$\|x\|_2 = \sqrt{\sum_{i=0}^{n-1} \chi_i^2}. \tag{6.2.1}$$

Notice that

$$\|x\|_2 \leq \sqrt{n} \max_{i=0}^{n-1} |\chi_i|$$

and hence, unless some χ_i is close to overflowing, the result will not overflow. The problem is that if some element χ_i has the property that χ_i^2 overflows, intermediate results in the computation in [\(6.2.1\)](#) will overflow. The solution is to determine k such that

$$|\chi_k| = \max_{i=0}^{n-1} |\chi_i|$$

and to then instead compute

$$\|x\|_2 = |\chi_k| \sqrt{\sum_{i=0}^{n-1} \left(\frac{\chi_i}{\chi_k}\right)^2}.$$

It can be argued that the same approach also avoids underflow if underflow can be avoided.

In our discussion, we mostly ignore this aspect of floating point computation.

Remark 6.2.1.3. Any time a real number is stored in our computer, it is stored as a nearby floating point number (element in F) (either through rounding or truncation). Nearby, of course, could mean that it is stored as the exact number if it happens to also be a floating point number.